

Exact elliptic witnesses

Complete rational division fibres and replayable independence lower bounds

Generalized Weierstrass arithmetic, rational model transport, two-isogenies, certified rational halving fibres and an independently implemented bounded certificate checker.

19

persistent mathematical object
kinds

11

worked scientific packets

2

independent rational witnesses
in the example

Complete rational halving fibres and witness-span lower bounds in bounded exact Python. No new Lean proof, complete Mordell-Weil basis, full saturation or global integer-point census.

Source, corpus receipts and reproduction commands accompany the repository snapshot.

What a caller can now do

The repository's polynomial machinery uses complete solution families with explicit inverse transport. This chapter adds a public rational elliptic arithmetic object to that workflow. A caller can register a nonsingular generalized Weierstrass model, add or multiply rational points, transport them between supported rationally isomorphic models, evaluate a rational two-isogeny, obtain every rational half of one point, and certify a lower bound for the rank of a supplied rational witness span modulo torsion. Definitions persist through SQLite restart and all operations have an explicit service allowlist.

The new `elliptic_curve` kind brings the catalogue to nineteen kinds. The corpus contains eleven scientific packets and a seven-request cold-reload service transcript. These are exact Python calculations and independently implemented Python checks. No new Lean kernel theorem, general rank algorithm, complete Mordell-Weil basis, full saturation or global integral-point census is claimed.

Exact generalized Weierstrass arithmetic

The model is $y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$, with five rational coefficients and nonzero discriminant. Infinity is JSON null. Coordinates use canonical rational strings in result packets; inputs can also use exact integers. Floating-point coefficients and coordinates are rejected.

Completing the square gives $Y = y + (a_1x + a_3)/2$ and $Y^2 = x^3 + (b_2/4)x^2 + (b_4/2)x + b_6/4$. Group arithmetic retains the original generalized coordinates. Addition checks its inputs and its output, includes inverse points and infinity, and scalar multiplication supports negative integers. Model transport completes the square, translates to short form and verifies a rational scaling of the short coefficients before mapping the point back. This supported scaling route finds rational model isomorphisms; it is not an arithmetic twist identification.

For example, the generalized model with a -invariants $(1, -1/4, 1, -1/2, -9/4)$ sends $(3, 3)$ to $(3, 5)$ on $Y^2 = x^3 - 2$. A second model $Y^2 = x^3 - 128$ receives $(12, 40)$ under scaling by two. Both inverse transports are tested. Rational two-isogenies require a checked nonidentity rational two-torsion kernel; the kernel and infinity map to infinity. The rational map and differential identity are retained in its packet.

Every rational half, including empty fibres

For a finite target with completed x -coordinate x_0 , write the completed cubic as $x^3 + A*x^2 + B*x + C$. The x -coordinates of its halves satisfy

$$\begin{aligned} x^4 - 4*x_0*x^3 - (2*B+4*x_0*A)*x^2 \\ - (8*C+4*x_0*B)*x + B^2-4*A*C-4*x_0*C = 0. \end{aligned}$$

This is the duplication formula after clearing its denominator. The checker also tests the completed cubic square and exact doubling, so a cleared-denominator root alone does not count as a half. The signs of the completed y -coordinate are both checked.

If one rational half H is discovered, every rational half is $H+T$ for a rational two-torsion point T , and every such sum is a half. Consequently one exact anchor together with the complete rational two-torsion subgroup certifies the whole fibre. The anchor may be found by bounded p -adic lifting and rational reconstruction; its validity is checked by exact doubling. This discovery route is not a completeness assumption. If it finds no anchor, a complete rational-root certificate covers the quartic, and every root is tested. Only that exhaustive certificate can justify an empty rational fibre. Over infinity, the fibre is exactly the rational two-torsion subgroup.

For a monic polynomial f of degree n , let D clear all coefficient denominators. Then $g(X)=D^n f(X/D)$ is monic and integral. Every rational root r gives a rational algebraic integer $D*r$, so $D*r$ is an integer. This reduces complete rational-root coverage to the existing signed-remainder Sturm certificate and its full integer subdivision tree. Repeated roots are retained through squarefree factorization. Replay checks the transform, factorization, signed remainders and tree; it does not discover roots again.

The pp-rational-halves/1 packet records the model, target, complete point list, anchor, complete two-torsion root certificate, any required quartic root certificate and shared node usage. Its `execution_verified` field is false. Exhausting the shared root budget raises `WorkLimit` and returns no partial complete answer.

On $y^2=x^3-2$, the point $(3,5)$ has no rational half, whereas its double has the single rational half $(3,5)$. On $y^2=x^3-x$, the fibre over infinity is infinity together with $(-1,0)$, $(0,0)$, $(1,0)$. On $y^2=x^3-25x$, doubling $(25/4, 75/8)$ produces a four-element rational fibre. The corpus retains all four cases.

Local characters as witness lower bounds

Let s clear the five model coefficient denominators. The integral cubic is

$$W^2 = F(X) = X^3 + s^2 b^2 X^2 + 8 s^4 b^4 X + 16 s^6 b^6, \\ X = 4 s^2 x, W = 8 s^3 Y.$$

At each retained odd good prime p , every root r of F modulo p defines a square-class character. If the point reduces to an ordinary finite point with X not equal to r , evaluate $X-r$. At the branch point $X=r$ use $F'(r)$. At infinity use the trivial square class. Encode a nonsquare as one and a square as zero.

The branch replacement is essential. From $W^2=(X-r)*G(X)$, a point approaching the branch has square class $X-r$ equal to the unit class $G(r)=F'(r)$. Nonintegral X has even negative valuation and square leading unit, so its character is zero. Primes dividing s are skipped: good reduction of the rational model alone does not ensure good reduction of the scaled integral model.

The group-law identity explains why these are homomorphisms. For a line $W=mX+b$ meeting the cubic in three ordinary points with coordinates X_1, X_2, X_3 , evaluation at r gives $(X_1-r)*(X_2-r)*(X_3-r)=(mr+b)^2$. Negation preserves X , hence the sum of two character bits is the bit of their group sum. At a branch point the derivative identity supplies the limiting square class. Tangent and vertical cases use the same group law and

the branch/infinity conventions. The tests independently enumerate several complete finite groups and verify the character identity for every pair, including their branch points and infinity.

The resulting binary matrix bounds the image dimension of the rational witnesses in $E(Q)/2E(Q)$. Rational odd torsion contributes no image; the total torsion image has dimension $t = \dim E(Q)[2]$. Thus the free rank of the witness span is at least $\text{matrix_rank} - t$, clipped to the number of witnesses. The packet can include additional exactly checked 2-primary torsion points to strengthen its finite matrix. It claims only this lower bound, never an upper bound or a basis.

Exact halving changes the lattice, preserving the rational span

An even relation in the finite character matrix can suggest a divisible combination. Discovery tests whether twice H equals a sum of working witnesses plus a checked torsion offset. Replacing one participating witness by H preserves their rational span modulo torsion: the replaced witness is twice H minus the other participating witnesses and the torsion offset. Conversely H is half that original combination modulo torsion. Each replacement is an invertible rational change of generators, not evidence that the original subgroup was already saturated.

The pp-elliptic-independence/2 packet records all original witnesses, the resulting working witnesses, exact replacement equations, auxiliary torsion, the complete local matrix, a proof of rational two-torsion dimension and the final lower bound. The torsion-dimension proof is either a root-free reduction of the monic cubic at a checked prime, or a complete rational-root certificate. Nonidentity auxiliary torsion and every offset satisfy an exact multiplication-by-sixteen identity.

The scientific example $y^2 = x^3 - 4x + 1$ has two independent witnesses $(0,1)$ and $(2,1)$: the character matrix already has rank two and rational two-torsion is absent. Starting with their doubles first gives trivial mod-two images; two recorded halving replacements recover the same rank-two lower bound. Duplicate, negated and torsion-only examples correctly fail to establish independence of every supplied witness.

Independence does not remove an inverse-coordinate restriction

The genus-two curve $y^2 = x^6 - 4x^2 + 1$ has first elliptic quotient $v^2 = u^3 - 4u + 1$ through $u = x^2$, $v = y$. The independent quotient point $(0,1)$ lifts to the integral source point $(0,1)$. The independent quotient point $(2,1)$ has no rational lift because two is not a rational square. Its independence certificate does not change that fact.

This example connects the new witness interface directly to the original perfect-power compiler: transport must retain the image condition. The existing `EllipticQuotientFamily.rational_lifts` returns the complete fibre over each specified quotient point, not a global point census. The new corpus includes both fibres alongside

the elliptic independence certificate.

Independent replay and explicit budgets

The new `elliptic_certificate_verifier.py` never calls the producer's `local_matrix`, `binary_rows`, `certify_independence` or halving discovery routines. It independently builds the integral cubic, enumerates good primes and roots, evaluates each local character and performs column-pivot binary Gaussian elimination. It shares exact group arithmetic, rational parsing and the existing Sturm certificate checker. Thus it guards against common errors in the producer's local-matrix or rank implementation, while retaining those shared Python routines as part of its trust boundary.

The checker requires exact schemas and canonical model/point coordinates. It rejects unknown guarantee fields, Boolean values in integer slots, malformed dimensions, unsupported versions, incorrect torsion proofs, altered local roots, forged halving equations and incorrect ranks. Certificate preflight limits nesting, lengths and integer bits before algebraic replay. A caller-supplied checking budget is independent of producer-reported node counts. Verification returns false on malformed input or exhausted checking work; the result does not silently become a weaker accepted certificate.

Producer independence queries accept at most 64 witnesses, prime bounds through 2000 and at most 64 halving replacements. Root-node limits are positive and at most 100000. Point coordinates are capped at 12000 bits, model coefficients at 8192 bits, and integer coefficients in new rational-root certificates at 12000 bits. The optional p-adic search is bounded, and the Sturm fallback is responsible for completeness when discovery fails. These are computational interface budgets, not universal bounds on elliptic arithmetic.

Public use and reproduction

```
from perfectpower.elliptic_arithmetic import EllipticCurve
from perfectpower.elliptic_certificate_verifier import verify_halves, verify_independence
```

```
E = EllipticCurve([0,-2])
doubled = E.point_multiply([3,5],2)
fibre = E.rational_halves(doubled)
assert verify_halves(fibre)

F = EllipticCurve([-4,1])
certificate = F.independence([[0,1],[2,1]],prime_bound=100)
assert certificate['rank_lower_bound'] == 2
assert verify_independence(certificate)
```

The catalogue exposes `summary`, `evidence`, `point_add`, `point_multiply`, `rational_halves`, `model_transport`, `two_isogeny` and `independence` on `elliptic_curve`. The standalone operations `verify_elliptic_halves` and `verify_elliptic_independence` accept a packet and optional checking limits. Private discovery and low-level group helpers remain outside the service allowlist.

```
make elliptic-witnesses
```

```

PYTHONPATH=python python python/develop_elliptic_witnesses.py --output /tmp/elliptic-replay
diff -r receipts/elliptic_witnesses /tmp/elliptic-replay
PYTHONPATH=python python -m perfectpower service --database /tmp/elliptic.sqlite \
  < receipts/elliptic_witnesses/service_requests.jsonl
make test
python python/render_polynomial_monograph.py --edition elliptic \
  --source docs/ELLIPTIC_WITNESSES_MONOGRAPH.md --output \
  docs/ELLIPTIC_WITNESSES_MONOGRAPH.pdf

```

Use a new database for deterministic service replay. Scientific receipts exclude wall-clock timings. The focused suites include independent finite-group enumeration, independent bounded rational-point comparisons, huge rational roots, generalized models, model-map round trips, isogeny exceptional fibres, exact halving changes with torsion offsets, malicious packets and cold persistence. Bounded enumeration is a differential test; completeness rests on the root certificate or the checked torsion-coset argument.

Remaining mathematical boundary

This push finishes a narrow public interface. It does not turn a rank lower bound into a global Mordell bound, discharge the rank-two logarithmic premise in the Thue machinery, give all rational points, certify a complete Mordell-Weil basis or prove global integral-point completeness. Arbitrary multiplication-division fibres, general isogeny kernels, complete saturation, Lean refinement of the interpreter, nonsplit stable reduction, general integral period markings and singular-endpoint analytic continuation remain separate work. The exact independent-witness example and complete rational division fibres are immediately usable without making those broader claims.